

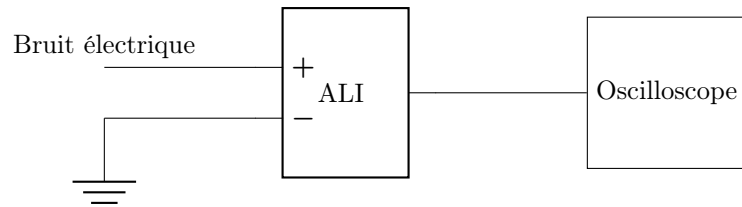
Annexes

June 8, 2026

Contents

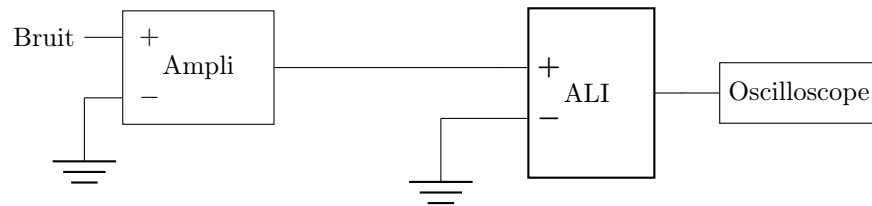
Historique des différents montage de l'Expérience	2
Programme 1 – Générateur aléatoire	5
generateur_aleatoire.h	5
generateur_aleatoire.c	7
Programme 2 – Marche aléatoire	12
marche_aleatoire.c	12
Programme 3 – Trouver les coefficients d'un générateur congruentiel	15
engine.h	15
main.c	18
fonction_generique.c	21
fonction_generation_aleatoire.c	24
gestion_donnee.c	25
algo_optimisation.c	26
multicoeur.c	29
fichier_test.c	32
Programme 4 – Test de la distance dans un carré	40
test_densite.c	40
Programme 5 – Analyse arduino	46
Fichier pour la carte arduino	46
Récupération du port série	47
Analyse du fichier renvoyer	48

Historique des différents montage de l'Expérience



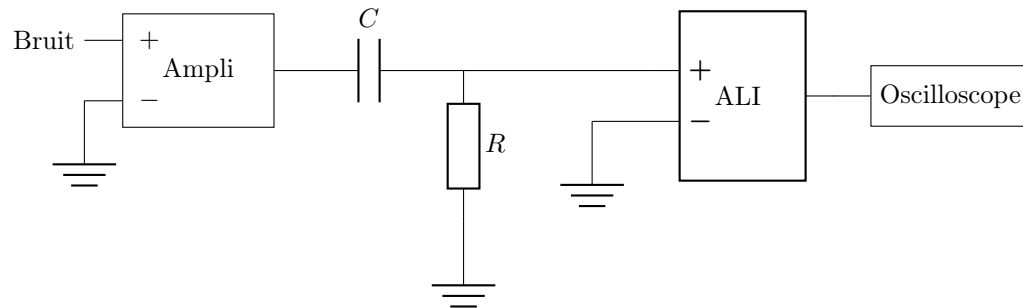
Description du problème :

- **Sortie** Trop faible pour être correctement observé.



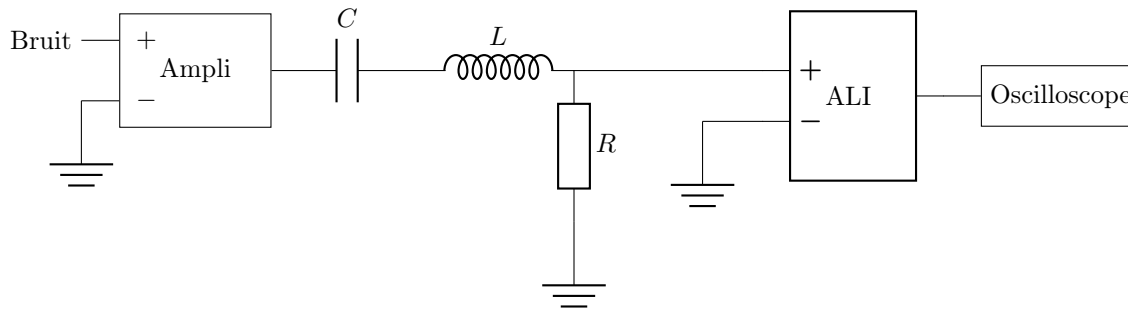
Description du problème :

- **Sortie** La présence du 50 Hz de EDF.



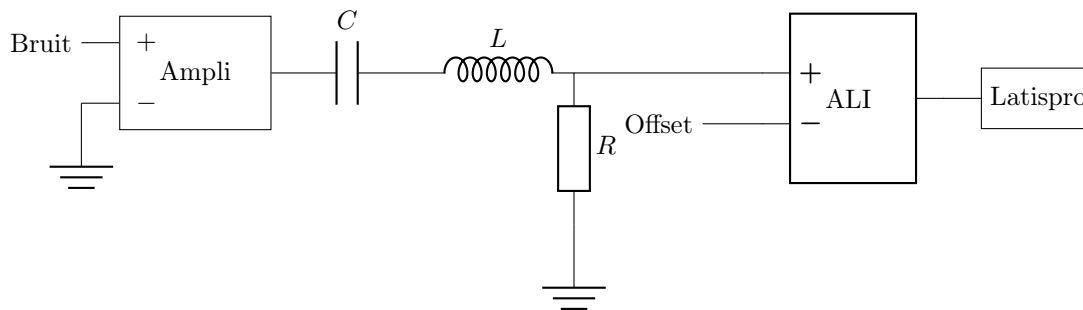
Description du problème :

- **Sortie** Signal presque constant car la fréquence max de notre signal dépasse celle de l'ALI.



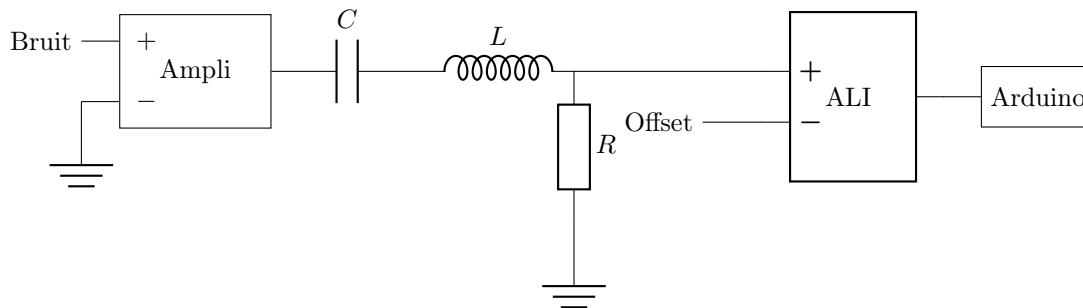
Description du problème :

- **Sortie** Le signal ne passe pas autant de temps en haut et bas, car les différents composant rajoute un offset, donc 0 n'est plus la moyenne du signal.



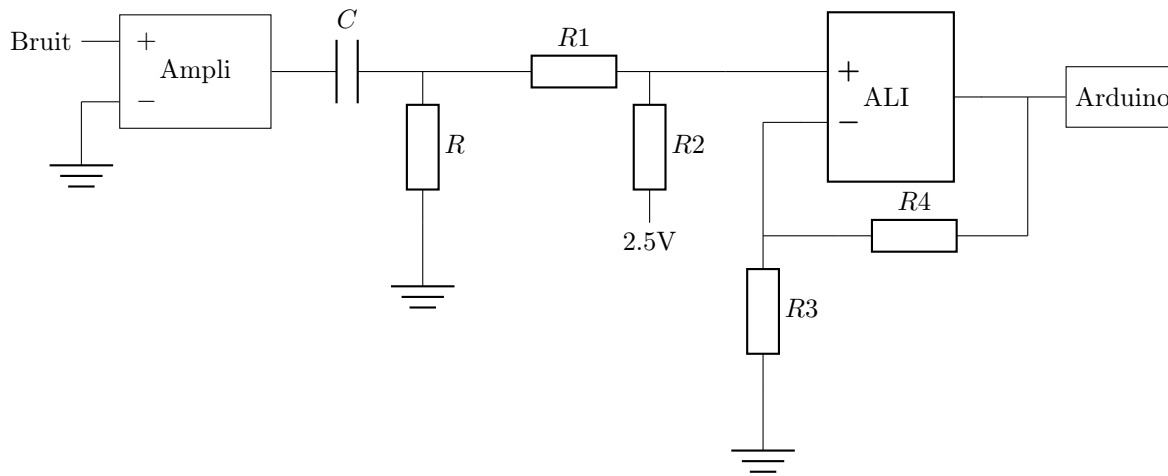
Description du problème :

- **Sortie** Aucun problème sur la sortie.
- **Vitesse d'acquisition** Vitesse d'acquisition beaucoup trop lente 3 600 000 points en 1h soit 125 000 nombres aléatoire. L'échantillon ne permet pas de test statistique viable.



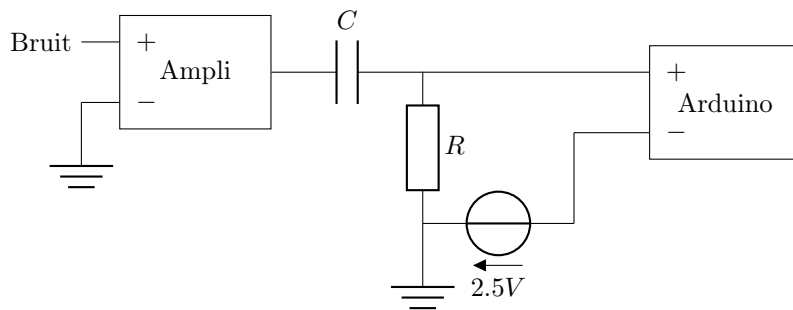
Description du problème :

- **Sortie** La tension de sortie est parfois négative, or arduino ne mesure que des tensions entre 0V et 5V.
- **Analyse** L'ALI peut être réalisé de manière informatique, et pose parfois des problèmes pendant l'acquisition.



Description du problème :

- **Sortie** Signal de sortie périodique, a cause du circuit d'addition.



Circuit Final :

- **Résistance** 3300 Ω .
- **Condensateur** 500 nF.
- **Donnée** 31 000 000 Millions de points en 2h, sur 2 jours différents.

Programme 1 – Générateur aléatoire

generateur_aleatoire.h

```
1 #include <stdlib.h>
2 #include <math.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <pthread.h>
6 #ifndef GENERATEUR_ALEATOIRE_H
7 #define GENERATEUR_ALEATOIRE_H
8
9 typedef struct _generateur_congruenciel_int{
10     int a;
11     int b;
12     int q;
13     int x;
14 } Generateur_congruenciel_int;
15
16 typedef struct _generateur_congruenciel_double{
17     double a;
18     double b;
19     double q;
20     double x;
21 } Generateur_congruenciel_double;
22
23 typedef struct _generateur_logistique {
24     double alpha;
25     double x;
26     int pas;
27 } Generateur_logistique;
28
29 typedef struct _generateur_physique {
30     double * t;
31     int i;
32     int i_max;
33 } Generateur_physique;
34
35 typedef struct _generateur_LSFR {
36     int * memoire;
37     int n_memoire;
38     int * polynome;
39     int n_polynome;
40 } Generateur_LSFR;
41
42 typedef struct _generateur_aleatoire{
```

```

43     Generateur_congruenciel_int * gcl_int; //1
44     Generateur_congruenciel_double * gcl_double; //2
45     Generateur_logistique * g_logistique; //3
46     Generateur_physique * g_physique; //4
47     Generateur_LSFR * g_lsfr; //5
48     int generateur_utiliser;
49 } Generateur_aleatoire;
50
51 Generateur_aleatoire * init_generateur(int generateur_utiliser);
52 void init_generateur_congruenciel_int(Generateur_aleatoire * g, int a, int b, int q, int seed);
53 void init_generateur_congruenciel_double(Generateur_aleatoire * g, double a, double b, double q, double seed);
54 void init_generateur_logistique(Generateur_aleatoire * g, double alpha, double seed, int pas);
55 void init_generateur_physique(Generateur_aleatoire * g, char * nom);
56 void init_generateur_LSFR(Generateur_aleatoire * g, int * bit_initial, int n_bit, int * polynome, int n_polynome);
57 double abs_double(double x);
58 double modulo_R_double(double x, double q);
59 double g_affine_modulo_double(Generateur_aleatoire * g);
60 double g_affine_modulo_int(Generateur_aleatoire * g);
61 double g_logistique(Generateur_aleatoire * g);
62 double g_physique(Generateur_aleatoire * g);
63 double g_lsfr(Generateur_aleatoire * g);
64 double g_aleatoire(Generateur_aleatoire * g);
65 void ecrire_tableau(char * nom, double * t, int n);
66
67 #endif

```

generateur_aleatoire.c

```
1 #include "generateur_aleatoire.h"
2 #include <stdio.h>
3
4 Generateur_aleatoire * init_generateur(int generateur_utiliser) {
5     Generateur_aleatoire * result = (Generateur_aleatoire *)malloc(sizeof(Generateur_aleatoire));
6     result->gcl_int = NULL;
7     result->gcl_double = NULL;
8     result->g_logistique = NULL;
9     result->g_physique = NULL;
10    result->generateur_utiliser = generateur_utiliser;
11    return result;
12 }
13
14 void init_generateur_congruenciel_int(Generateur_aleatoire * g, int a, int b, int q, int seed) {
15     Generateur_congruenciel_int * gcl = (Generateur_congruenciel_int *)malloc(sizeof(Generateur_congruenciel_int));
16     gcl->a = a;
17     gcl->b = b;
18     gcl->q = q;
19     gcl->x = seed;
20     g->gcl_int = gcl;
21 }
22
23 void init_generateur_congruenciel_double(Generateur_aleatoire * g, double a, double b, double q, double seed) {
24     Generateur_congruenciel_double * gcl = (Generateur_congruenciel_double *)malloc(sizeof(Generateur_congruenciel_double));
25     gcl->a = a;
26     gcl->b = b;
27     gcl->q = q;
28     gcl->x = seed;
29     g->gcl_double = gcl;
30 }
31
32 void init_generateur_logistique(Generateur_aleatoire * g, double alpha, double seed, int pas) {
33     Generateur_logistique * g_l = (Generateur_logistique *)malloc(sizeof(Generateur_logistique));
34     g_l->alpha = alpha;
35     g_l->x = seed;
36     g_l->pas = pas;
37     g->g_logistique = g_l;
38 }
39
40 void init_generateur_physique(Generateur_aleatoire * g, char * nom) {
41     FILE * f = fopen(nom, "r");
42     if (f == NULL) {
43         printf("Erreur de lecture de fichier dans init_generateur_physique\n");
44         return;
45     }
46 }
```

```

45     }
46     char tampom[1000];
47     int i_max = 0;
48     while (fgets(tampom, 1000, f) != NULL) {
49         i_max ++;
50     }
51     fclose(f);
52     double * t = (double *)malloc(i_max*sizeof(double));
53     f = fopen(nom, "r");
54     if (f == NULL) {
55         printf("Erreur de lecture de fichier dans init_generateur_physique\n");
56         return;
57     }
58     int i = 0;
59     char * end;
60     while (fgets(tampom, 1000, f) != NULL) {
61         t[i] = strtod(tampom, &end);
62         i++;
63     }
64     fclose(f);
65     Generateur_physique * g_p = (Generateur_physique *)malloc(sizeof(Generateur_physique));
66     g_p->i = 0;
67     g_p->i_max = i_max-1;
68     g_p->t = t;
69     g->g_physique = g_p;
70 }
71
72 void init_generateur_LSFR(Generateur_aleatoire * g, int * bit_initial, int n_bit, int * polynome, int n_polynome) {
73     Generateur_LSFR * g_lsfr = (Generateur_LSFR *)malloc(sizeof(Generateur_LSFR));
74     g_lsfr->n_memoire = n_bit;
75     g_lsfr->n_polynome = n_polynome;
76     g_lsfr->memoire = bit_initial;
77     g_lsfr->polynome = polynome;
78     g->g_lsfr = g_lsfr;
79 }
80
81 double abs_double(double x) {
82     if (x<0){
83         return -x;
84     }
85     return x;
86 }
87
88 double modulo_R_double(double x, double q) {
89     double y = 0;
90     while (y<abs_double(x)) {

```



```

91     y += q;
92 }
93
94 if (x>=0) {
95     return abs_double(x) - y + q;
96 }
97 else {
98     return abs_double(abs_double(x) - y);
99 }
100 }
101
102 double g_affine_modulo_double(Generateur_aleatoire * g) {
103     double result = modulo_R_double((g->gcl_double->a*g->gcl_double->x+g->gcl_double->b), g->gcl_double->q);
104     g->gcl_double->x = result;
105     return result;
106 }
107
108 double g_affine_modulo_int(Generateur_aleatoire * g) {
109     int result = modulo_R_double((g->gcl_int->a*g->gcl_int->x+g->gcl_int->b), g->gcl_int->q);
110     g->gcl_int->x = result;
111     return (double)result/(double)g->gcl_int->q;
112 }
113
114 double g_logistique(Generateur_aleatoire * g) {
115     double result;
116     for (int i = 0; i<g->g_logistique->pas; i++) {
117         result = g->g_logistique->alpha*g->g_logistique->x*(1 - g->g_logistique->x);
118         g->g_logistique->x = result;
119     }
120     return (2.0 / M_PI) * asin(sqrt(result));
121 }
122
123 double g_physique(Generateur_aleatoire * g) {
124     double result = g->g_physique->t[g->g_physique->i];
125     if (g->g_physique->i >= g->g_physique->i_max) {
126         g->g_physique->i = 0;
127     }
128     else {
129         g->g_physique->i += 1;
130     }
131     return result;
132 }
133
134 double g_lsfr(Generateur_aleatoire * g) {
135     int * valeur_bin = (int *)malloc(63*sizeof(int));
136     int s = 0;

```

```

137     for (int i = 0; i<63; i++) {
138         s = 0;
139         for (int j = 0; j<g->g_lsfr->n_polynome; j++) {
140             s += g->g_lsfr->memoire[g->g_lsfr->polynome[j]];
141         }
142         s = s % 2;
143         valeur_bin[i] = g->g_lsfr->memoire[g->g_lsfr->n_memoire-1];
144         for (int j = g->g_lsfr->n_memoire; j > 0; j--) {
145             g->g_lsfr->memoire[j] = g->g_lsfr->memoire[j-1];
146         }
147         g->g_lsfr->memoire[0] = s;
148     }
149     ulong v = 0;
150     ulong m = 0;
151     for (int i = 0; i<63; i++) {
152         v = 2*v + valeur_bin[i];
153         m = 2*m + 1;
154     }
155     return (double)v/((double)m;
156 }
157
158 double g_aleatoire(Generateur_aleatoire * g) {
159     if (g->generateur_utiliser == 2) {
160         return g_affine_modulo_double(g);
161     }
162     if (g->generateur_utiliser == 1) {
163         return g_affine_modulo_int(g);
164     }
165     if (g->generateur_utiliser == 3) {
166         return g_logistique(g);
167     }
168     if (g->generateur_utiliser == 4) {
169         return g_physique(g);
170     }
171     if (g->generateur_utiliser == 5) {
172         return g_lsfr(g);
173     }
174     printf("Pas de generateur defini\n");
175     exit(1);
176 }
177
178 void ecrire_tableau(char * nom, double * t, int n) {
179     FILE * f = fopen(nom, "w");
180     for (int i = 0; i<n; i++) {
181         fprintf(f, "%lf\n", t[i]);
182     }

```

```
183 |     fclose(f);  
184 | }
```

Programme 2 – Marche aléatoire

marche_aleatoire.c

```
1 #include "../Programme_generation_aleatoire_C/generateur_aleatoire.h"
2 #include <raylib.h>
3 #include <stdio.h>
4 #include <time.h>
5
6 typedef struct _particule {
7     double x;
8     double y;
9 } Particule;
10
11 typedef struct _univers {
12     Particule ** tab;
13     int nb;
14 } Univers;
15
16 void object_render(Univers * u) {
17     double radius = 1;
18     for (int i = 0; i < u->nb; i++) {
19         DrawCircle(u->tab[i]->x, u->tab[i]->y, radius, RAYWHITE);
20     }
21 }
22
23 void render(Camera2D camera, Univers * u) {
24     BeginDrawing();
25     ClearBackground(BLACK);
26     BeginMode2D(camera);
27
28     object_render(u);
29
30     EndMode2D();
31     EndDrawing();
32 }
33
34 Univers * init_univers(double x, double y, int nb) {
35     Particule ** t = (Particule **)malloc(nb*sizeof(Particule *));
36     for (int i = 0; i < nb; i++) {
37         t[i] = (Particule *)malloc(sizeof(Particule));
38         t[i]->x = x;
39         t[i]->y = y;
40     }
41     Univers * u = (Univers *)malloc(sizeof(Univers));
42     u->nb = nb;
```

```

43     u->tab = t;
44     return u;
45 }
46
47 void mouvement_aleatoire(Univers * u, Generateur_aleatoire * g) {
48     double alpha;
49     double v = 1;
50     for (int i = 0; i < u->nb; i++) {
51         alpha = g_aleatoire(g) * 2*M_PI;
52         u->tab[i]->x += v*cos(alpha);
53         u->tab[i]->y += v*sin(alpha);
54     }
55 }
56
57 void afficher_variable(int variable, double x, double y) {
58     char * str = malloc(20*sizeof(char));
59     sprintf(str, "%d", variable);
60     DrawText(str, x, y, 50, RAYWHITE);
61 }
62
63
64 int main(void){
65     Camera2D camera = {0};
66     int width = 600;
67     int height = width;
68     InitWindow(width, height, "Marche_aleatoire");
69     camera.target = (Vector2){0, 0}; // le centre du cercle
70     camera.offset = (Vector2){width/2.0f, height/2.0f}; // centre de l'écran
71     camera.zoom = 2; // zoom normal
72     camera.rotation = 0;
73     struct timespec ts;
74     ts.tv_sec = 0; // secondes
75     ts.tv_nsec = 10 * 1000000; // nanosecondes -> 500 ms
76
77     Univers * u = init_univers(0, 0, 10000);
78     Generateur_aleatoire * g = init_generateur(4);
79     init_generateur_congruenciel_int(g, 23, 19, 60006, 55);
80     //init_generateur_congruenciel_int(g, 374218, 28460991, 68822490, 55);
81
82     init_generateur_logistique(g, 4, 0.10932, 10);
83     //init_generateur_physique(g, "Donnee/liste_nb_aleatoire_physique.txt");
84     init_generateur_physique(g, "liste_nb_aleatoire_physique.txt");
85
86     int * memoire_lsfr = (int *)malloc(16*sizeof(int));
87     int * polynome_lsfr = (int *)malloc(5*sizeof(int));
88     memoire_lsfr[0] = 0;

```

```

89     memoire_lsfr[1] = 0;
90     memoire_lsfr[2] = 0;
91     memoire_lsfr[3] = 1;
92     memoire_lsfr[4] = 0;
93     memoire_lsfr[5] = 0;
94     memoire_lsfr[6] = 0;
95     memoire_lsfr[7] = 1;
96     memoire_lsfr[8] = 1;
97     memoire_lsfr[9] = 1;
98     memoire_lsfr[10] = 0;
99     memoire_lsfr[11] = 0;
100    memoire_lsfr[12] = 1;
101    memoire_lsfr[13] = 0;
102    memoire_lsfr[14] = 0;
103    memoire_lsfr[15] = 1;
104
105    polynome_lsfr[0] = 0;
106    polynome_lsfr[1] = 10;
107    polynome_lsfr[2] = 12;
108    polynome_lsfr[3] = 13;
109    polynome_lsfr[4] = 15;
110    init_générateur_LSFR(g, memoire_lsfr, 16, polynome_lsfr, 5);
111    //1=affine_module_int 2=affini_modulo_double 3=logistique 4=physique 5=lsfr
112
113    int frame = 0;
114    while (!WindowShouldClose()) {
115        render(camera, u);
116        afficher_variable(frame, 100, 150);
117        mouvement_aleatoire(u, g);
118        if (frame == 41) {
119            TakeScreenshot("marche_aleatoire_congruentiel1.png");
120        }
121        if (frame == 101) {
122            TakeScreenshot("marche_aleatoire_congruentiel2.png");
123        }
124        if (frame == 401) {
125            TakeScreenshot("marche_aleatoire_congruentiel3.png");
126        }
127        if (frame == 1001) {
128            TakeScreenshot("marche_aleatoire_congruentiel4.png");
129        }
130        nanosleep(&ts, NULL);
131        frame++;
132    }
133    return 0;
134 }

```

Programme 3 – Trouver les coefficients d’un générateur congruentiel

engine.h

```
1 #ifndef ENGINE_H_
2 #define ENGINE_H_
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <stdbool.h>
7 #include <math.h>
8 #include <pthread.h>
9
10 typedef struct retour_optimisation_AM_ {
11     double a;
12     double b;
13     double cout;
14 } retour_optimisation_AM;
15
16 typedef struct parametre_reseaux_AM_ {
17     double a;
18     double b;
19     double q;
20     double * X;
21     double * Y_reel;
22     double learning_rate;
23     int n;
24 } parametre_reseaux_AM;
25
26 typedef struct parametre_reseaux_multicoeur_AM_ {
27     parametre_reseaux_AM * P;
28     int nb_generation;
29 } parametre_reseaux_multicoeur_AM;
30
31 typedef struct intervalle_ {
32     double a_min;
33     double a_max;
34     double b_min;
35     double b_max;
36 } intervalle;
37
38 /*
39 #####
40 fonction_generation_aleatoire.c
41 #####
42 */
```

```

43 double modulo_R(double x, double q);
44 double affine_modulo(double a, double b, double q, double x);
45
46 /*
47 #####
48             fonction_generique.c
49 #####
50 */
51 double abs_double(double x);
52 double distance(double x1, double y1, double x2, double y2);
53 void afficher_tableau_linespace(double ** X, int n);
54 intervalle * initialisation_intervalle(double a_min, double a_max, double b_min, double b_max);
55 retour_optimisation_AM * initialisation_retour_optimisation_AM(double a_initial, double b_initial, double cout);
56 parametre_reseaux_AM * initialisation_parametre_reseaux_AM(double a, double b, double q, double * X, double * Y_reel, double
    learning_rate, int n);
57 double ** linespace(intervalle * I, int n);
58 void supprimer_tableau(double ** X, int n);
59 void supprimer_donnee_AM(retour_optimisation_AM ** X, int n);
60 retour_optimisation_AM * find_min_AM(retour_optimisation_AM ** tableau, int n);
61 retour_optimisation_AM * find_max_AM(retour_optimisation_AM ** tableau, int n);
62
63
64 /*
65 #####
66             algo_optimisation.c
67 #####
68 */
69 double * forward_AM(double a, double b, double q, double * X, int n);
70 double fun_cout_AM(double * Y_reel, double * Y_final, int n);
71 double d_fun_cout_AM(double * Y_reel, double * Y_final, int n);
72 double d_a_AM(double x);
73 double d_b_AM();
74 double d_fun_cout_a_AM(double Y_reel, double Y_final, double x, int n);
75 double d_fun_cout_b_AM(double * Y_reel, double * Y_final, int n);
76 retour_optimisation_AM * descente_gradient_AM(parametre_reseaux_AM * P);
77 retour_optimisation_AM * apprentissage_AM(parametre_reseaux_AM * P, int nb_generation);
78 intervalle * reduction_intervalle_AM(retour_optimisation_AM ** donnee, int n, intervalle * initial);
79
80 /*
81 #####
82             multicoeur.c
83 #####
84 */
85 parametre_reseaux_multicoeur_AM * initialisation_parametre_reseaux_multicoeur_AM(double a, double b, double q, double * X, double *
    Y_reel, double learning_rate, int n, int nb_generation);
86 void * apprentissage_multicoeur_AM(void *p_data);

```



```

87 retour_optimisation_AM ** calcul_multicoeur_AM(intervalle * I, int n_linespace, double q, double * X, double * Y_reel, int taille,
88     double learning_rate, int nb_generation, int nb_coeur);
89
90 /*
91 #####
92     gestion_donnee.c
93 #####
94 */
95 void tableau_cout_inf_X_sauvegarde(char * nom, double X, retour_optimisation_AM ** donnee, int n);
96 void ecrire_chemin_descente_gradient(char * nom, double a, double b);
97 retour_optimisation_AM ** tableau_cout_inf_X_tableau(int *tableau_fin, double X, retour_optimisation_AM ** donnee, int n);
98
99 /*
100 #####
101     fichier_test.c
102 #####
103 */
104 void verifier(void);
105 #endif // ENGINE_H_

```

main.c

```
1 #include "engine.h"
2 #include <stdio.h>
3 #include <time.h>
4
5 int main(void) {
6     verifier();
7     clock_t start, end;
8     double cpu_time_used;
9
10    start = clock();
11
12    int taille = 30;
13    double q = 9854547;
14    double * X = (double *)malloc(taille*sizeof(double));
15    double * Y_reel = (double *)malloc(taille*sizeof(double));
16    double a = 3580131;
17    double b = 2766521;
18    for(int i = 0; i<taille; i++) {
19        X[i] = i;
20        Y_reel[i] = affine_modulo(a, b, q, i);
21    }
22    double cherche_autour_a = 3580131.111508;
23    double cherche_autour_b = 2766519.522335;
24    double carre = 100;
25    intervalle * I = initialisation_intervalle(cherche_autour_a - carre, cherche_autour_a + carre, cherche_autour_b - carre,
        cherche_autour_b + carre);
26    retour_optimisation_AM ** resultat;
27    int n_linespace = 2000;
28    double learning_rate = 0.000000005;
29    int nb_generation = 10;
30    int nb_coeur = 16;
31    for (int i = 0; i<0; i++) {
32        start = clock();
33        resultat = calcul_multicoeur_AM(I, n_linespace, q, X, Y_reel, taille, learning_rate, nb_generation, nb_coeur);
34        retour_optimisation_AM * min = find_min_AM(resultat, n_linespace*n_linespace);
35        printf("a = %lf, b = %lf, cout = %lf\n", min->a, min->b, min->cout);
36        I = reduction_intervalle_AM(resultat, n_linespace*n_linespace, I);
37        printf("a_min = %lf, a_max = %lf, b_min = %lf, b_max = %lf\n", I->a_min, I->a_max, I->b_min, I->b_max);
38        end = clock();
39        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
40        printf("Temps CPU : %.4f secondes\n", cpu_time_used);
41    }
42    //resultat = calcul_multicoeur_AM(I, n_linespace, q, X, Y_reel, taille, learning_rate, nb_generation, nb_coeur);
43}
```

```

44  /*
45  double ratio = 0.25;
46  retour_optimisation_AM * min = find_min_AM(resultat, n_linespace*n_linespace);
47  retour_optimisation_AM * max = find_max_AM(resultat, n_linespace*n_linespace);
48  double cout_opti = (max->cout - min->cout)*0.5 + min->cout;
49  tableau_cout_inf_X_sauvegarde("donnee_all3.txt", max->cout, resultat, n_linespace*n_linespace);
50  //tableau_cout_inf_X_sauvegarde("donnee1.txt", cout_opti, resultat, n_linespace*n_linespace);
51  cout_opti = (max->cout - min->cout)*ratio + min->cout;
52  printf("cout_opti = %lf\n", cout_opti);
53  //tableau_cout_inf_X_sauvegarde("donnee2.txt", cout_opti, resultat, n_linespace*n_linespace);
54  cout_opti = (max->cout - min->cout)*0.15 + min->cout;
55  //tableau_cout_inf_X_sauvegarde("donnee3.txt", cout_opti, resultat, n_linespace*n_linespace);
56  cout_opti = (max->cout - min->cout)*0.05 + min->cout;
57  //tableau_cout_inf_X_sauvegarde("donnee4.txt", cout_opti, resultat, n_linespace*n_linespace);
58  cout_opti = (max->cout - min->cout)*0.005 + min->cout;
59  //tableau_cout_inf_X_sauvegarde("donnee5.txt", cout_opti, resultat, n_linespace*n_linespace);
60  cout_opti = max->cout;
61  //tableau_cout_inf_X_sauvegarde("donnee6.txt", cout_opti, resultat, n_linespace*n_linespace);
62  */
63
64  int test = 0;
65  int nb_test = 10000;
66  double * valeur_relle = (double *)malloc(nb_test*sizeof(double));
67  double * valeur_trouver = (double *)malloc(nb_test*sizeof(double));
68  double tolerance = 1000;
69  double init_X = ((double)rand()/((double)RAND_MAX)*(q-1);
70  for (int i = 0; i<nb_test; i++) {
71      init_X = ((double)rand()/((double)RAND_MAX)*(q-1);
72      valeur_relle[i] = affine_modulo(a, b, q, init_X);
73      valeur_trouver[i] = affine_modulo(3580130.998952, 2766521.010579, q, init_X);
74      if (i%(nb_test/100) == 0) {
75          printf("i = %d valeur %lf trouver %lf ecart %lf\n", i, valeur_relle[i], valeur_trouver[i], abs_double(valeur_relle[i]-
              valeur_trouver[i]));
76      }
77      if (abs_double(valeur_relle[i]-valeur_trouver[i])<tolerance) {
78          test += 1;
79      }
80  }
81  printf("réussit %lf\n", (double)test/((double)nb_test);
82  /*
83  parametre_reseaux_AM * P = initialisation_parametre_reseaux_AM(3580031, 2766621, q, X, Y_reel, learning_rate, taille);
84  retour_optimisation_AM * calcul;
85  double cout = 4406279991699.031250;
86  double last_cout = cout;
87  int i = 0;
88  while (cout>10) {

```

```

89     calcul = descente_gradient_AM(P);
90     P->a = calcul->a;
91     P->b = calcul->b;
92     cout = calcul->cout;
93     if (cout + 1 < last_cout) {
94         ecrire_chemin_descente_gradient("chemin_descente_gradient.txt", P->a, P->b = calcul->b);
95         last_cout = cout;
96         printf("cout = %lf\n", cout);
97     }
98     free(calcul);
99 }
100 */
101 return 0;
102 }

```

fonction_generique.c

```
1 #include "engine.h"
2
3 double abs_double(double x) {
4     if (x <= 0) {
5         return -x;
6     }
7     else {
8         return x;
9     }
10 }
11
12 double distance(double x1, double y1, double x2, double y2) {
13     return sqrt((x2-x1)*(x2-x1) + (y2-y1)*(y2-y1));
14 }
15
16 void afficher_tableau_linespace(double ** X, int n) {
17     for (int i = 0; i < n; i++) {
18         printf("X[%d] = [%lf, %lf]\n", i, X[i][0], X[i][1]);
19     }
20 }
21
22 intervalle * initialisation_intervalle(double a_min, double a_max, double b_min, double b_max) {
23     intervalle * I = (intervalle *)malloc(sizeof(intervalle));
24     I->a_min = a_min;
25     I->a_max = a_max;
26     I->b_min = b_min;
27     I->b_max = b_max;
28     return I;
29 }
30
31 retour_optimisation_AM * initialisation_retour_optimisation_AM(double a_initial, double b_initial, double cout) {
32     retour_optimisation_AM * retour = (retour_optimisation_AM *)malloc(sizeof(retour_optimisation_AM));
33     retour->a = a_initial;
34     retour->b = b_initial;
35     retour->cout = cout;
36     return retour;
37 }
38
39 parametre_reseaux_AM * initialisation_parametre_reseaux_AM(double a, double b, double q, double * X, double * Y_reel, double
    learning_rate, int n) {
40     parametre_reseaux_AM * retour = (parametre_reseaux_AM *)malloc(sizeof(parametre_reseaux_AM));
41     retour->a = a;
42     retour->b = b;
43     retour->q = q;
```

```

44     retour->X = X;
45     retour->Y_reel = Y_reel;
46     retour->learning_rate = learning_rate;
47     retour->n = n;
48     return retour;
49 }
50
51 double ** linespace(intervalle * I, int n) {
52     // n est le cote du carre
53     double ** X = (double **)malloc(n*n*sizeof(double *));
54     for (int i = 0; i<n*n; i++) {
55         X[i] = (double *)malloc(2*sizeof(double));
56     }
57     double a = I->a_min;
58     double b = I->b_min;
59     double ecart_a = (I->a_max - I->a_min)/(n-1);
60     double ecart_b = (I->b_max - I->b_min)/(n-1);
61     for (int i = 0; i<n; i++) {
62         double b = I->b_min;
63         for (int j = 0; j<n; j++) {
64             X[i*n+j][0] = a;
65             X[i*n+j][1] = b;
66             b += ecart_b;
67         }
68         a += ecart_a;
69     }
70     return X;
71 }
72
73 void supprimer_tableau(double ** X, int n) {
74     for (int i = 0; i<n; i++) {
75         free(X[i]);
76     }
77     free(X);
78 }
79
80 void supprimer_donnee_AM(retour_optimisation_AM ** X, int n) {
81     for (int i = 0; i<n; i++) {
82         free(X[i]);
83     }
84     free(X);
85 }
86
87 retour_optimisation_AM * find_min_AM(retour_optimisation_AM ** tableau, int n) {
88     retour_optimisation_AM * resultat = tableau[0];
89     for (int i = 0; i<n; i++) {

```

```

90         if (tableau[i]->cout < resultat->cout) {
91             resultat = tableau[i];
92         }
93     }
94     return resultat;
95 }

96
97 retour_optimisation_AM * find_max_AM(retour_optimisation_AM ** tableau, int n) {
98     retour_optimisation_AM * resultat = tableau[0];
99     for (int i = 0; i < n; i++) {
100         if (tableau[i]->cout > resultat->cout) {
101             resultat = tableau[i];
102         }
103     }
104     return resultat;
105 }

```

fonction_generation_aleatoire.c

```
1 #include "engine.h"
2
3 double modulo_R(double x, double q) {
4     double y = 0;
5     while (y < abs_double(x)) {
6         y += q;
7     }
8
9     if (x >= 0) {
10         return abs_double(x) - y + q;
11     }
12     else {
13         return abs_double(abs_double(x) - y);
14     }
15 }
16
17 double affine_modulo(double a, double b, double q, double x) {
18     return modulo_R((a*x+b), q);
19 }
```


gestion_donnee.c

```
1 #include "engine.h"
2 #include <stdio.h>
3
4 void tableau_cout_inf_X_sauvegarde(char * nom, double X, retour_optimisation_AM ** donnee, int n) {
5     FILE * f = fopen(nom, "a");
6     for (int i = 0; i < n; i++) {
7         if (donnee[i]->cout < X) {
8             fprintf(f, "a = %lf; b = %lf; cout = %lf\n", donnee[i]->a, donnee[i]->b, donnee[i]->cout);
9         }
10    }
11    fclose(f);
12 }
13
14 void ecrire_chemin_descente_gradient(char * nom, double a, double b) {
15     FILE * f = fopen(nom, "a");
16     fprintf(f, "a = %lf; b = %lf\n", a, b);
17     fclose(f);
18 }
19
20 retour_optimisation_AM ** tableau_cout_inf_X_tableau(int *tableau_fin, double X, retour_optimisation_AM ** donnee, int n) {
21     retour_optimisation_AM ** t = (retour_optimisation_AM **)malloc(n*sizeof(retour_optimisation_AM *));
22     *tableau_fin = 0;
23     for (int i = 0; i < n; i++) {
24         if (donnee[i]->cout < X) {
25             t[*tableau_fin] = donnee[i];
26             *tableau_fin += 1;
27         }
28     }
29     return t;
30 }
```

algo_optimisation.c

```
1 #include "engine.h"
2
3 /*
4  AM = affine_modulo
5  */
6
7 double * forward_AM(double a, double b, double q, double * X, int n) {
8     double * Y = (double *)malloc(n*sizeof(double));
9     for (int i = 0; i<n; i++) {
10         Y[i] = modulo_R(a*X[i]+b, q);
11     }
12     return Y;
13 }
14
15 double fun_cout_AM(double * Y_reel, double * Y_final, int n) {
16     double somme = 0;
17     for (int i = 0; i<n; i++) {
18         somme += (Y_reel[i]-Y_final[i])*(Y_reel[i]-Y_final[i]);
19     }
20     return somme/((double)n;
21 }
22
23 double d_fun_cout_AM(double * Y_reel, double * Y_final, int n) {
24     double somme = 0;
25     for (int i = 0; i<n; i++) {
26         somme += -2*(Y_reel[i]-Y_final[i]);
27     }
28     return somme/n;
29 }
30
31 double d_a_AM(double x) {
32     return x;
33 }
34
35 double d_b_AM() {
36     return 1;
37 }
38
39 double d_fun_cout_a_AM(double Y_reel, double Y_final, double x, int n) {
40     return (2*(Y_reel-Y_final)/n)*d_a_AM(x);
41 }
42
43 double d_fun_cout_b_AM(double * Y_reel, double * Y_final, int n) {
44     return d_fun_cout_AM(Y_reel, Y_final, n)*d_b_AM();
```

```

45 }
46
47 retour_optimisation_AM * descente_gradient_AM(parametre_reseaux_AM * P) {
48     double * Y_final = forward_AM(P->a, P->b, P->q, P->X, P->n);
49     double grad_a = 0;
50     double grad_b = d_fun_cout_b_AM(Y_final, P->Y_reel, P->n);
51     for (int i = 0; i < P->n; i++) {
52         grad_a += d_fun_cout_a_AM(Y_final[i], P->Y_reel[i], P->X[i], P->n);
53     }
54     retour_optimisation_AM * resultat = initialisation_retour_optimisation_AM(P->a - grad_a * P->learning_rate, P->b - grad_b * P->
        learning_rate, fun_cout_AM(P->Y_reel, Y_final, P->n));
55     free(Y_final);
56     return resultat;
57 }
58
59 retour_optimisation_AM * apprentissage_AM(parametre_reseaux_AM * P, int nb_generation) {
60     retour_optimisation_AM * calcul;
61     double cout = -1;
62     for (int i = 0; i < nb_generation; i++) {
63         calcul = descente_gradient_AM(P);
64         P->a = calcul->a;
65         P->b = calcul->b;
66         cout = calcul->cout;
67         ecrire_chemin_descente_gradient("chemin_descente_gradient.txt", P->a, P->b = calcul->b);
68         free(calcul);
69     }
70     retour_optimisation_AM * retour = initialisation_retour_optimisation_AM(P->a, P->b, cout);
71     return retour;
72 }
73
74 intervalle * reduction_intervalle_AM(retour_optimisation_AM ** donnee, int n, intervalle * initial) {
75     double ratio = 0.25;
76
77     intervalle * final = (intervalle *) malloc(sizeof(intervalle));
78     retour_optimisation_AM * min = find_min_AM(donnee, n);
79     retour_optimisation_AM * max = find_max_AM(donnee, n);
80     double cout_opti = min->cout * ratio + min->cout;
81     double dx = (initial->a_max - initial->a_min) / 2;
82     double dy = (initial->b_max - initial->b_min) / 2;
83     int taille_tableau_inf_X;
84     double d_max = distance(0, 0, dx, dy);
85     retour_optimisation_AM ** tableau_inf_X = tableau_cout_inf_X_tableau(&taille_tableau_inf_X, cout_opti, donnee, n);
86     double * nb_voisin = (double *) malloc(taille_tableau_inf_X * sizeof(double));
87     for (int i = 0; i < taille_tableau_inf_X; i++) {
88         nb_voisin[i] = 0;
89     }

```

```

90
91 //calculer le nb de voisin pour chaque point dans le nouvelle intervalle
92 for (int i = 0; i<taille_tableau_inf_X; i++) {
93     for (int j = 0; j<taille_tableau_inf_X; j++) {
94         if (distance(tableau_inf_X[i]->a, tableau_inf_X[i]->b, tableau_inf_X[j]->a, tableau_inf_X[j]->b) < d_max) {
95             nb_voisin[i] += 1;
96         }
97     }
98 }
99 //trouver le point qui a le plus de voisin dans le nouvelle intervalle
100 int i_max = 0;
101 for (int i = 0; i<taille_tableau_inf_X; i++) {
102     if (nb_voisin[i] > nb_voisin[i_max]) {
103         i_max = i;
104     }
105 }
106 final->a_min = tableau_inf_X[i_max]->a - dx/2;
107 if (final->a_min<initial->a_min) {
108     final->a_min = initial->a_min;
109 }
110 final->a_max = tableau_inf_X[i_max]->a + dx/2;
111 if (final->a_max>initial->a_max) {
112     final->a_max = initial->a_max;
113 }
114 final->b_min = tableau_inf_X[i_max]->b - dy/2;
115 if (final->b_min<initial->b_min) {
116     final->b_min = initial->b_min;
117 }
118 final->b_max = tableau_inf_X[i_max]->b + dy/2;
119 if (final->b_max>initial->b_max) {
120     final->b_max = initial->b_max;
121 }
122
123 supprimer_donnee_AM(donnee, n);
124 free(tableau_inf_X);
125 free(nb_voisin);
126 free(initial);
127 return final;
128 }

```

multicoeur.c

```
1 #include "engine.h"
2 #include <stdio.h>
3
4 parametre_reseaux_multicoeur_AM * initialisation_parametre_reseaux_multicoeur_AM(double a, double b, double q, double * X, double *
   Y_reel, double learning_rate, int n, int nb_generation) {
5     parametre_reseaux_multicoeur_AM * retour = (parametre_reseaux_multicoeur_AM *)malloc(sizeof(parametre_reseaux_multicoeur_AM));
6     retour->P = initialisation_parametre_reseaux_AM(a, b, q, X, Y_reel, learning_rate, n);
7     retour->nb_generation = nb_generation;
8     return retour;
9 }
10
11 void * apprentissage_multicoeur_AM(void *p_data) {
12     parametre_reseaux_multicoeur_AM * P_multicoeur = (parametre_reseaux_multicoeur_AM *)p_data;
13     parametre_reseaux_AM * P = P_multicoeur->P;
14     int nb_generation = P_multicoeur->nb_generation;
15
16     retour_optimisation_AM * calcul;
17     double cout = -1;
18     for (int i = 0; i < nb_generation; i++) {
19         calcul = descente_gradient_AM(P);
20         P->a = calcul->a;
21         P->b = calcul->b;
22         cout = calcul->cout;
23         free(calcul);
24     }
25     retour_optimisation_AM * retour = initialisation_retour_optimisation_AM(P->a, P->b, cout);
26     return (void *)retour;
27 }
28
29 retour_optimisation_AM ** calcul_multicoeur_AM(intervalle * I, int n_linespace, double q, double * X, double * Y_reel, int taille,
   double learning_rate, int nb_generation, int nb_coeur) {
30     retour_optimisation_AM ** resultat = (retour_optimisation_AM **)malloc(n_linespace*n_linespace*sizeof(retour_optimisation_AM *));
31     double ** X_a_b = linespace(I, n_linespace);
32     pthread_t * tableau_coeur = (pthread_t *)malloc(nb_coeur*sizeof(pthread_t));
33     parametre_reseaux_multicoeur_AM ** tableau_parametre_reseaux_multicoeur_AM = (parametre_reseaux_multicoeur_AM **)malloc(nb_coeur*
   sizeof(parametre_reseaux_multicoeur_AM *));
34
35     double progression = 0.01;
36     for (int i = 0; i < (n_linespace*n_linespace)/nb_coeur; i++) {
37         if ((double)i/((double)n_linespace*n_linespace/nb_coeur) >= progression) {
38             progression += 0.01;
39             printf("calcul effectuer a %lf%%\n", (double)i/((double)n_linespace*n_linespace/nb_coeur)*100);
40         }
41         //allocation mémoire
```

```

42     for (int j = 0; j<nb_coeur; j++) {
43         tableau_parametre_reseaux_multicoeur_AM[j] = initialisation_parametre_reseaux_multicoeur_AM(X_a_b[i*nb_coeur+j][0], X_a_b[
44             i*nb_coeur+j][1], q, X, Y_reel, learning_rate, taille, nb_generation);
45     }
46
47     //allocation calcul
48     for (int j = 0; j<nb_coeur; j++) {
49         pthread_create(&tableau_coeur[j], NULL, apprentissage_multicoeur_AM, (void *)tableau_parametre_reseaux_multicoeur_AM[j]);
50     }
51
52     //recuperation resultat
53     for (int j = 0; j<nb_coeur; j++) {
54         pthread_join(tableau_coeur[j], (void**)&resultat[i*nb_coeur+j]);
55     }
56
57     //supprimer memoire
58     for (int j = 0; j<nb_coeur; j++) {
59         free(tableau_parametre_reseaux_multicoeur_AM[j]->P);
60         free(tableau_parametre_reseaux_multicoeur_AM[j]);
61     }
62
63     //gestion reste des calcul quand pas divisible par nb_coeur
64
65     int i = (n_linespace*n_linespace)/nb_coeur;
66     //allocation memoire
67     for (int j = 0; j<(n_linespace*n_linespace)%nb_coeur; j++) {
68         tableau_parametre_reseaux_multicoeur_AM[j] = initialisation_parametre_reseaux_multicoeur_AM(X_a_b[i*nb_coeur+j][0], X_a_b[i*
69             nb_coeur+j][1], q, X, Y_reel, learning_rate, taille, nb_generation);
70     }
71
72     //allocation calcul
73     for (int j = 0; j<(n_linespace*n_linespace)%nb_coeur; j++) {
74         pthread_create(&tableau_coeur[j], NULL, apprentissage_multicoeur_AM, (void *)tableau_parametre_reseaux_multicoeur_AM[j]);
75     }
76
77     //recuperation resultat
78     for (int j = 0; j<(n_linespace*n_linespace)%nb_coeur; j++) {
79         pthread_join(tableau_coeur[j], (void**)&resultat[i*nb_coeur+j]);
80     }
81
82     //supprimer memoire
83     for (int j = 0; j<(n_linespace*n_linespace)%nb_coeur; j++) {
84         free(tableau_parametre_reseaux_multicoeur_AM[j]->P);
85         free(tableau_parametre_reseaux_multicoeur_AM[j]);
86     }

```

```
86  
87     return resultat;  
88 }
```

fichier_test.c

```
1 #include "engine.h"
2
3 bool verifier_modulo_R(void) {
4     bool resultat = true;
5     double tolerance = pow(10,-10);
6
7     double x = 12.7;
8     double q = 3.5;
9     double y = modulo_R(x, q);
10    double y_reel = 2.2;
11
12    if ((y <= y_reel - tolerance)|| (y >= y_reel + tolerance)) {
13        resultat = false;
14        printf("Erreur modulo_R(12.7, 3.5) != %lf valeur = %lf\n", y_reel, y);
15    }
16
17    x = -117;
18    q = 17;
19    y = modulo_R(x, q);
20    y_reel = 2;
21
22    if ((y <= y_reel - tolerance)|| (y >= y_reel + tolerance)) {
23        resultat = false;
24        printf("Erreur modulo_R(-117, 17) != %lf valeur = %lf\n", y_reel, y);
25    }
26
27    x = -5.3;
28    q = 4.3;
29    y = modulo_R(x, q);
30    y_reel = 3.3;
31
32    if ((y <= y_reel - tolerance)|| (y >= y_reel + tolerance)) {
33        resultat = false;
34        printf("Erreur modulo_R(-5.3, 4.3) != %lf valeur = %lf\n", y_reel, y);
35    }
36
37    return resultat;
38 }
39
40 bool verifier_abs_double(void) {
41     bool resultat = true;
42     double tolerance = pow(10,-10);
43
44     double x = 5.6;
```



```

45     double y = abs_double(x);
46     double y_reel = 5.6;
47
48     if ((y <= y_reel - tolerance) || (y >= y_reel + tolerance)) {
49         resultat = false;
50         printf("Erreur abs_double(5.6) != %lf valeur = %lf\n", y_reel, y);
51     }
52
53     x = -5.6;
54     y = abs_double(x);
55     y_reel = 5.6;
56
57     if ((y <= y_reel - tolerance) || (y >= y_reel + tolerance)) {
58         resultat = false;
59         printf("Erreur abs_double(-5.6) != %lf valeur = %lf\n", y_reel, y);
60     }
61     return resultat;
62 }
63
64 bool verifier_affine_modulo(void) {
65     bool resultat = true;
66     double tolerance = pow(10, -10);
67
68     double x = 12.7;
69     double a = 2.1;
70     double b = 8.6;
71     double q = 3.5;
72     double y = affine_modulo(a, b, q, x);
73     double y_reel = 0.27;
74
75     if ((y <= y_reel - tolerance) || (y >= y_reel + tolerance)) {
76         resultat = false;
77         printf("Erreur affine_modulo(2.1, 8.6, 3.5, 12.7) != %lf valeur = %lf\n", y_reel, y);
78     }
79
80     x = -12.7;
81     a = 2.1;
82     b = 8.6;
83     q = 3.5;
84     y = affine_modulo(a, b, q, x);
85     y_reel = 2.93;
86
87     if ((y <= y_reel - tolerance) || (y >= y_reel + tolerance)) {
88         resultat = false;
89         printf("Erreur affine_modulo(2.1, 8.6, 3.5, -12.7) != %lf valeur = %lf\n", y_reel, y);
90     }

```

```

91     return resultat;
92 }
93
94
95 bool verifier_forward_AM(void) {
96     bool resultat = true;
97     double tolerance = pow(10,-10);
98
99     double * X = (double *)malloc(3*sizeof(double));
100     X[0] = 5;
101     X[1] = 2.6;
102     X[2] = -13.6;
103     double a = 2.1;
104     double b = 8.6;
105     double q = 3.5;
106     int n = 3;
107     double * y = forward_AM(a, b, q, X, n);
108     double * Y_reel = (double *)malloc(3*sizeof(double));
109     Y_reel[0] = 1.6;
110     Y_reel[1] = 0.06;
111     Y_reel[2] = 1.04;
112
113     for (int i = 0; i<n; i++) {
114         if ((y[i] <= Y_reel[i] - tolerance)|| (y[i] >= Y_reel[i] + tolerance)) {
115             resultat = false;
116             printf("Erreur affine_modulo(2.1, 8.6, 3.5, %lf) != %lf valeur = %lf\n", X[i], Y_reel[i], y[i]);
117         }
118     }
119     free(X);
120     free(Y_reel);
121     free(y);
122     return resultat;
123 }
124
125 bool verifier_fun_cout_AM(void) {
126     bool resultat = true;
127     double tolerance = pow(10,-10);
128
129     double * X1 = (double *)malloc(3*sizeof(double));
130     X1[0] = 5;
131     X1[1] = 2.6;
132     X1[2] = -13.6;
133     double * X2 = (double *)malloc(3*sizeof(double));
134     X2[0] = -3;
135     X2[1] = 7.9;
136     X2[2] = -9.06;

```

```

137     int n = 3;
138     double y = fun_cout_AM(X1, X2, n);
139     double y_reel = 37.5672;
140
141     if ((y <= y_reel - tolerance) || (y >= y_reel + tolerance)) {
142         resultat = false;
143         printf("Erreur fun_cout_AM([5,2.6,-13.6], [-3,7.9,-9.06], 3) != %lf valeur = %lf\n", y_reel, y);
144     }
145
146     free(X1);
147     free(X2);
148     return resultat;
149 }
150
151 bool verifier_d_fun_cout_AM(void) {
152     bool resultat = true;
153     double tolerance = pow(10,-10);
154
155     double * X1 = (double *)malloc(3*sizeof(double));
156     X1[0] = 5;
157     X1[1] = 2.6;
158     X1[2] = -13.6;
159     double * X2 = (double *)malloc(3*sizeof(double));
160     X2[0] = -3;
161     X2[1] = 7.9;
162     X2[2] = -9.06;
163     int n = 3;
164     double y = d_fun_cout_AM(X1, X2, n);
165     double y_reel = 1.2266666666;
166
167     if ((y <= y_reel - tolerance) || (y >= y_reel + tolerance)) {
168         resultat = false;
169         printf("Erreur d_fun_cout_AM([5,2.6,-13.6], [-3,7.9,-9.06], 3) != %lf valeur = %lf\n", y_reel, y);
170     }
171
172     free(X1);
173     free(X2);
174     return resultat;
175 }
176
177 bool verifier_descent_gradient_AM(void) {
178     bool resultat = true;
179     double tolerance = pow(10,-10);
180
181     double * X1 = (double *)malloc(3*sizeof(double));
182     X1[0] = 5;

```

```

183 X1[1] = 2.6;
184 X1[2] = -13.6;
185 double * X2 = (double *)malloc(3*sizeof(double));
186 X2[0] = -3;
187 X2[1] = 7.9;
188 X2[2] = -9.06;
189 int n = 3;
190 double a = 4.6;
191 double b = -5.2;
192 double q = 7.2;
193 double learning_rate = pow(10, -5);
194 parametre_reseaux_AM * P = initialisation_parametre_reseaux_AM(a, b, q, X1, X2, learning_rate, n);
195 retour_optimisation_AM * y = descente_gradient_AM(P);
196 retour_optimisation_AM * Y_reel = initialisation_retour_optimisation_AM(4.60101229333333261451, -5.1998762667 , 73.049866666666688);
197
198 if ((y->a <= Y_reel->a - tolerance)|| (y->a >= Y_reel->a + tolerance)) {
199     resultat = false;
200     printf("Erreur descente_gradient_AM(4.6, -5.2, 7.2, [5,2.6,-13.6], [-3,7.9,-9.06], 10**-5, 3)->a != %lf valeur = %.20lf\n",
201           Y_reel->a, y->a);
202 }
203 if ((y->b <= Y_reel->b - tolerance)|| (y->b >= Y_reel->b + tolerance)) {
204     resultat = false;
205     printf("Erreur descente_gradient_AM(4.6, -5.2, 7.2, [5,2.6,-13.6], [-3,7.9,-9.06], 10**-5, 3)->b != %lf valeur = %lf\n",
206           Y_reel->b, y->b);
207 }
208 if ((y->cout <= Y_reel->cout - tolerance)|| (y->cout >= Y_reel->cout + tolerance)) {
209     resultat = false;
210     printf("Erreur descente_gradient_AM(4.6, -5.2, 7.2, [5,2.6,-13.6], [-3,7.9,-9.06], 10**-5, 3)->cout != %lf valeur = %lf\n",
211           Y_reel->cout, y->cout);
212 }
213
214 free(X1);
215 free(X2);
216 free(y);
217 free(Y_reel);
218 free(P);
219 return resultat;
220 }
221
222 bool verifier_apprentissage_AM(void) {
223     bool resultat = true;
224     double tolerance = pow(10, -10);
225
226     double * X1 = (double *)malloc(3*sizeof(double));
227     X1[0] = 5;
228     X1[1] = 2.6;

```

```

226 X1[2] = -13.6;
227 double * X2 = (double *)malloc(3*sizeof(double));
228 X2[0] = -3;
229 X2[1] = 7.9;
230 X2[2] = -9.06;
231 int n = 3;
232 double a = 4.6;
233 double b = -5.2;
234 double q = 7.2;
235 double learning_rate = pow(10, -5);
236 parametre_reseaux_AM * P = initialisation_parametre_reseaux_AM(a, b, q, X1, X2, learning_rate, n);
237 retour_optimisation_AM * y = apprentissage_AM(P, 100);
238 retour_optimisation_AM * Y_reel = initialisation_retour_optimisation_AM(4.6943428505, -5.1878058630, 81.476294322750618);
239
240 if ((y->a <= Y_reel->a - tolerance)|| (y->a >= Y_reel->a + tolerance)) {
241     resultat = false;
242     printf("Erreur apprentissage_AM(4.6, -5.2, 7.2, [5,2.6,-13.6], [-3,7.9,-9.06], 10**-5, 3)->a != %lf valeur = %.10lf\n", Y_reel
        ->a, y->a);
243 }
244 if ((y->b <= Y_reel->b - tolerance)|| (y->b >= Y_reel->b + tolerance)) {
245     resultat = false;
246     printf("Erreur apprentissage_AM(4.6, -5.2, 7.2, [5,2.6,-13.6], [-3,7.9,-9.06], 10**-5, 3)->b != %lf valeur = %.10lf\n", Y_reel
        ->b, y->b);
247 }
248
249 free(X1);
250 free(X2);
251 free(y);
252 free(Y_reel);
253 free(P);
254 return resultat;
255 }
256
257 bool verifier_linespace(void) {
258     bool resultat = true;
259     double tolerance = pow(10, -10);
260
261     intervalle * I = initialisation_intervalle(1, 2, 1, 2);
262     double ** X = linespace(I, 3);
263     double ** Y = (double **)malloc(3*3*sizeof(double *));
264     for (int i = 0; i<3*3; i++) {
265         Y[i] = (double *)malloc(2*sizeof(double));
266     }
267     Y[0][0] = 1;
268     Y[0][1] = 1;
269     Y[1][0] = 1;

```

```

270 Y[1][1] = 1.5;
271 Y[2][0] = 1;
272 Y[2][1] = 2;
273 Y[3][0] = 1.5;
274 Y[3][1] = 1;
275 Y[4][0] = 1.5;
276 Y[4][1] = 1.5;
277 Y[5][0] = 1.5;
278 Y[5][1] = 2;
279 Y[6][0] = 2;
280 Y[6][1] = 1;
281 Y[7][0] = 2;
282 Y[7][1] = 1.5;
283 Y[8][0] = 2;
284 Y[8][1] = 2;
285
286 for (int i = 0; i<3*3; i++) {
287     for (int j = 0; j<2; j++) {
288         if ((X[i][j] <= Y[i][j] - tolerance)|| (X[i][j] >= Y[i][j] + tolerance)) {
289             resultat = false;
290             printf("Erreur linespace([1,2,1,2], 3)[%d][%d] != %lf valeur = %lf\n", i, j, X[i][j], Y[i][j]);
291         }
292     }
293 }
294
295 free(I);
296 supprimer_tableau(X, 3*3);
297 supprimer_tableau(Y, 3*3);
298 return resultat;
299 }
300
301 void verifier(void) {
302     bool resultat = true;
303     resultat &= verifier_abs_double();
304     resultat &= verifier_modulo_R();
305     resultat &= verifier_affine_modulo();
306
307     resultat &= verifier_forward_AM();
308     resultat &= verifier_fun_cout_AM();
309     resultat &= verifier_d_fun_cout_AM();
310     /*
311     les fonction d_a_AM, d_b_AM, d_fun_cout_a_AM, d_fun_cout_b_AM, initialisation_retour_optimisation_AM,
312         initialisation_parametre_reseaux_AM sont correct car leur calcul est evident(elles sont la pour faciliter la lecture)
313     */
314     resultat &= verifier_descent_gradient_AM();
315     resultat &= verifier_apprentissage_AM();

```

```
315
316     resultat &= verifier_linespace();
317
318     if (resultat == true) {
319         printf("Tout va bien\n");
320     }
321 }
```

Programme 4 – Test de la distance dans un carré

test_densite.c

```
1 #include "../Programme_generation_aleatoire_C/generateur_aleatoire.h"
2 #include <stdio.h>
3 #include <stdbool.h>
4
5 typedef struct _fonction {
6     double * Y;
7     double * t;
8     int n;
9 } Fonction;
10
11 typedef struct _point {
12     double x;
13     double y;
14 } Point;
15
16 typedef struct _parametre_multicoeur {
17     double * nb_aleatoire;
18     int n;
19     double x_min;
20     double x_max;
21     int nb_interval;
22     Fonction * retour;
23 } Parametre_multicoeur;
24
25 void supprimer_fonction(Fonction * f) {
26     free(f->Y);
27     free(f->t);
28     free(f);
29 }
30
31 Fonction * addition_fonction(Fonction * f, Fonction *g) {
32     double * y = (double *)malloc(f->n*sizeof(double));
33     double * t = (double *)malloc(f->n*sizeof(double));
34     Fonction * r = (Fonction *)malloc(sizeof(Fonction));
35     for (int i = 0; i<f->n; i++) {
36         y[i] = f->Y[i] + g->Y[i];
37         t[i] = f->t[i];
38     }
39     r->Y = y;
40     r->t = t;
41     r->n = f->n;
42     return r;
```



```

43 }
44
45 Fonction * soustraction_fonction(Fonction * f, Fonction *g) {
46     double * y = (double *)malloc(f->n*sizeof(double));
47     double * t = (double *)malloc(f->n*sizeof(double));
48     Fonction * r = (Fonction *)malloc(sizeof(Fonction));
49     for (int i = 0; i<f->n; i++) {
50         y[i] = f->Y[i] - g->Y[i];
51         t[i] = f->t[i];
52     }
53     r->Y = y;
54     r->t = t;
55     r->n = f->n;
56     return r;
57 }
58
59 Fonction * init_fonction(double x_min, double x_max, int n) {
60     double * y = (double *)malloc((n+1)*sizeof(double));
61     double * t = (double *)malloc((n+1)*sizeof(double));
62     double dt = (x_max-x_min)/(n);
63     Fonction * r = (Fonction *)malloc(sizeof(Fonction));
64     for (int i = 0; i<n+1; i++) {
65         y[i] = 0;
66         t[i] = i*dt;
67     }
68     r->Y = y;
69     r->t = t;
70     r->n = n+1;
71     return r;
72 }
73
74 void afficher_tableau(double * X, int n) {
75     for (int i = 0; i<n; i++) {
76         printf("%lf\n", X[i]);
77     }
78 }
79
80 double produit_scalaire(Fonction * f_X, Fonction * f_Y) {
81     double aire = 0;
82     double dt = f_X->t[1];
83     for (int i = 0; i<f_X->n; i++) {
84         aire += f_X->Y[i]*f_Y->Y[i]*dt;
85     }
86     return aire;
87 }
88

```

```

89 double distance(Point P1, Point P2) {
90     return (P1.x-P2.x)*(P1.x-P2.x)+(P1.y-P2.y)*(P1.y-P2.y);
91 }
92
93 Fonction * approche_densiter(double * nb_aleatoire, int n, double x_min, double x_max, int nb_interval) {
94     double * p_interval = (double *)malloc((nb_interval+1) * sizeof(double));
95     double * t = (double *)malloc((nb_interval+1)*sizeof(double));
96     Fonction * f = (Fonction *)malloc(sizeof(Fonction));
97     f->Y = p_interval;
98     f->t = t;
99     f->n = nb_interval+1;
100     double dt = (x_max-x_min)/(double) nb_interval;
101     int n_liste = (n/2);
102     Point * liste_point = (Point *)malloc( n_liste* sizeof(Point));
103     double nb_distance = (double)n_liste*((double)n_liste-1)/2;
104     for (int i = 0; i<n_liste; i++) {
105         liste_point[i].x = nb_aleatoire[2*i];
106         liste_point[i].y = nb_aleatoire[2*i+1];
107     }
108     for (int i = 0; i<nb_interval+1; i++) {
109         t[i] = i*dt;
110         p_interval[i] = 0;
111     }
112     int k;
113     for (int i = 0; i<n_liste; i++) {
114         for (int j = i+1; j<n_liste; j++) {
115             k = floor(distance(liste_point[i], liste_point[j])/dt);
116             p_interval[k] += 1/(nb_distance*dt);
117         }
118     }
119     free(liste_point);
120     return f;
121 }
122
123 Fonction * densiter_theorique(double x_min, double x_max, int nb_interval) {
124     double * Y = (double *)malloc((nb_interval+1) * sizeof(double));
125     double * t = (double *)malloc((nb_interval+1)*sizeof(double));
126     double dt = (x_max-x_min)/(double)(nb_interval);
127     Fonction * f = (Fonction *)malloc(sizeof(Fonction));
128     for (int i = 0; i<nb_interval+1; i++) {
129         t[i] = i*dt;
130         if (t[i]<1) {
131             Y[i] = M_PI + t[i] - 4*sqrt(t[i]);
132         }
133         else {
134             Y[i] = -2 + 4*asin(1/(sqrt(t[i]))) + 4*sqrt(t[i]-1) - M_PI - t[i];

```

```

135     }
136 }
137 }
138 f->Y = Y;
139 f->t = t;
140 f->n = nb_interval+1;
141 return f;
142 }
143
144 Parametre_multicoeur * init_parametre_multicoeur(int n, double x_min, double x_max, int nb_interval, Generateur_aleatoire * g) {
145     Parametre_multicoeur * p = (Parametre_multicoeur *)malloc(sizeof(Parametre_multicoeur));
146     double * nb_aleatoire = (double *)malloc(n*sizeof(double));
147     for (int i = 0; i<n; i++) {
148         nb_aleatoire[i] = g_aleatoire(g);
149     }
150     p->nb_aleatoire = nb_aleatoire;
151     p->n = n;
152     p->x_min = x_min;
153     p->x_max = x_max;
154     p->nb_interval = nb_interval;
155     p->retour = NULL;
156     return p;
157 }
158
159 void supprimer_parametre_multicoeur(Parametre_multicoeur * p) {
160     free(p->nb_aleatoire);
161     supprimer_fonction(p->retour);
162     free(p);
163 }
164
165 void * calcul_approche_densiter_multicoeur(void * p_data) {
166     Parametre_multicoeur * parametre = (Parametre_multicoeur *)p_data;
167     Fonction * densiter = approche_densiter(parametre->nb_aleatoire, parametre->n, parametre->x_min, parametre->x_max, parametre->
        nb_interval);
168     parametre->retour = densiter;
169     return NULL;
170 }
171
172 Fonction * approche_densiter_multicoeur(double x_min, double x_max, int len_liste_nb_aleatoire, int nb_interval, int nb_coeur, int
    nb_iteration, Generateur_aleatoire * g) {
173     Parametre_multicoeur ** liste_parametre_multicoeur = (Parametre_multicoeur **)malloc(nb_coeur*sizeof(Parametre_multicoeur *));
174     pthread_t * tableau_coeur = (pthread_t *)malloc(nb_coeur*sizeof(pthread_t));
175     Fonction * resultat = init_fonction(x_min, x_max, nb_interval);
176     Fonction * f_temp;
177     for (int i = 0; i<nb_iteration; i++) {
178         printf("Generation %d\n", i);

```

```

179 //allocation memoire
180 for (int j = 0; j<nb_coeur; j++) {
181     liste_parametre_multicoeur[j] = init_parametre_multicoeur(len_liste_nb_aleatoire, x_min, x_max, nb_interval, g);
182 }
183 //allocation calcul
184 for (int j = 0; j<nb_coeur; j++) {
185     pthread_create(&tableau_coeur[j], NULL, calcul_approche_densiter_multicoeur, (void *)liste_parametre_multicoeur[j]);
186 }
187
188 //recuperation resultat
189 for (int j = 0; j<nb_coeur; j++) {
190     pthread_join(tableau_coeur[j], NULL);
191 }
192
193 for (int j = 0; j<nb_coeur; j++) {
194     f_temp = addition_fonction(resultat, liste_parametre_multicoeur[j]->retour);
195     supprimer_fonction(resultat);
196     supprimer_parametre_multicoeur(liste_parametre_multicoeur[j]);
197     resultat = f_temp;
198 }
199 }
200 for (int i = 0; i<nb_interval+1; i++) {
201     resultat->Y[i] = resultat->Y[i]/(double)(nb_iteration*nb_coeur);
202 }
203 return resultat;
204 }
205
206 double approche_densiter_multicoeur_esperance(double x_min, double x_max, int len_liste_nb_aleatoire, int nb_interval, int nb_coeur,
207 int nb_iteration, Generateur_aleatoire * g) {
208     Parametre_multicoeur ** liste_parametre_multicoeur = (Parametre_multicoeur **)malloc(nb_coeur*sizeof(Parametre_multicoeur *));
209     pthread_t * tableau_coeur = (pthread_t *)malloc(nb_coeur*sizeof(pthread_t));
210     Fonction * f_th = densiter_theorique(x_min, x_max, nb_interval);
211     double esperance = 0;
212     double esperance_du_carre = 0;
213     double nb_calcul =(double)(nb_iteration*nb_coeur);
214     Fonction * temp;
215     for (int i = 0; i<nb_iteration; i++) {
216         printf("Generation %d\n", i);
217         //allocation memoire
218         for (int j = 0; j<nb_coeur; j++) {
219             liste_parametre_multicoeur[j] = init_parametre_multicoeur(len_liste_nb_aleatoire, x_min, x_max, nb_interval, g);
220         }
221         //allocation calcul
222         for (int j = 0; j<nb_coeur; j++) {
223             pthread_create(&tableau_coeur[j], NULL, calcul_approche_densiter_multicoeur, (void *)liste_parametre_multicoeur[j]);

```

```

224
225 //recuperation resultat
226 for (int j = 0; j<nb_coeur; j++) {
227     pthread_join(tableau_coeur[j], NULL);
228 }
229
230 for (int j = 0; j<nb_coeur; j++) {
231     temp = soustraction_fonction(liste_parametre_multicoeur[j]->retour, f_th);
232     double valeur = produit_scalaire(temp, temp);
233     esperance += valeur/nb_calcul;
234     esperance_du_carre += valeur*valeur/nb_calcul;
235     supprimer_fonction(temp);
236     supprimer_parametre_multicoeur(liste_parametre_multicoeur[j]);
237 }
238 }
239 printf("esperance final = %.30lf\n", esperance);
240 printf("esperance du carre final = %.30lf\n", esperance_du_carre);
241 return esperance;
242 }
243
244 int main(void) {
245     //1=affine_module_int 2=affini_modulo_double 3=logistique 4=physique
246     Generateur_aleatoire * g = init_generateur(1);
247     init_generateur_congruenciel_int(g, 65539, 0, 1073741824, 78);
248     //init_generateur_congruenciel_int(g, 23, 19, 60006, 55);
249     init_generateur_logistique(g, 4, 0.8921, 30);
250     init_generateur_physique(g, "liste_nb_aleatoire_physique.txt");
251     //x_min, x_max len_liste_nb_aleaoire nb_intervalle nb_coeur nb_rep generateur
252     //Fonction * d = approche_densiter_multicoeur(0, 2, 10000, 10000, 4, 10, g);
253     //ecrire_tableau("test.txt", d->Y, d->n);
254     double esperance = approche_densiter_multicoeur_esperance(0, 2, 100000, 100000, 4, 10, g);
255     return 0;
256 }

```

Programme 5 – Analyse arduino

Fichier pour la carte arduino

```
1 void setup() {
2   Serial.begin(2000000); //utilise un moniteur série de 2000000 de bit seconde
3 }
4
5 void loop() {
6   int val = analogRead(A1); //recupère la tension
7
8   //on affiche toujours des nombre avec 4 caractère
9   if (val <= -1) {
10    Serial.println("0000"); //traite le cas 0
11  }
12  else {
13    if (val < 10) {
14      Serial.print("0"); //pour ajouter des zero devant en cas de pb
15    }
16    if (val < 100) {
17      Serial.print("0"); //pour ajouter des zero devant en cas de pb
18    }
19    if (val < 1000) {
20      Serial.print("0"); //pour ajouter des zero devant en cas de pb
21    }
22  }
23  Serial.println(val);
24 }
```

Récupération du port série

```
1 import serial
2 import time
3 from datetime import datetime
4
5 # set up the serial line
6 sensor="pile"
7 serial_port = '/dev/ttyACM0'
8 baud_rate=2000000
9 ser = serial.Serial(serial_port, baud_rate)
10 time.sleep(1)
11
12 path = "/media/msoucaze/Partage/Data{}.txt"
13 t_start = time.time()
14 nb_par_fichier = 100000
15 nb_fichier = 1
16 initial = 0
17
18 for i in range(initial, nb_fichier+initial):
19     output_file = open(path.format(i), "w+")
20     for j in range(nb_par_fichier):
21         line = ser.readline()
22         line = line.strip()
23         if (len(line) != 4):
24             print("Pb dans le moniteur série", line, j, "fichier", i) #affiche une alerte en cas de différence de longueur dans le
                moniteur série
25         try:
26             line_str = str(line, 'UTF-8')
27             output_file.writelines([line_str, "\n"])
28         except:
29             pass
30
31     print("fin fichier ", i)
32     output_file.close()
33
34 t_final = time.time()
35 output_file.close()
36 print("temp execution {}s".format(t_final-t_start)) #affiche le temps d'acquisition
```

Analyse du fichier renvoyer

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 def liste_01_nombre(liste, i, nb_valeur, pas):
5     p_2 = 2**((nb_valeur-1)
6     n = 0
7     for j in range(i, i+nb_valeur*pas, pas):
8         n += p_2*liste[j]
9         p_2 = p_2/2
10    return n
11
12 #emplacement = 'courbe_TIPE/V2/3_{}.txt'
13 emplacement = '/media/msoucaze/Partage/acquisition_final2/Data{}.txt'
14 nom_sortie = 'liste_nb_aleatoire_physique.txt'
15 nom_sortie_binaire_txt = 'liste_bin.txt'
16 nom_sortie_binaire_bin = 'liste_bin.bin'
17 nb_prise = 10
18 barre = 0
19 nb_01_interpretation = 32
20 pas = 1
21 T = []
22 A = []
23 liste_01 = []
24 liste_nb_aleatoire = []
25 liste_nb_aleatoire_uniforme = []
26 nb_elt = 0
27 sortie_txt = True
28 sortie_binaire_txt = True
29 sortie_binaire_bin = False
30
31
32 for i in range(nb_prise):
33     f = open(emplacement.format(i), 'r', encoding="latin-1")
34     contenu = f.readlines()
35
36     for line in contenu:
37         try:
38             A.append(float(line))
39             nb_elt += 1
40         except:
41             pass
42     f.close()
43
44 A = np.array(A)
```



```

45 barre = np.mean(A)
46 nb_1 = 0
47 nb_0 = 0
48
49
50 for i in range(len(A)):
51     if A[i] > barre:
52         liste_01.append(1)
53         nb_1 += 1
54     else:
55         liste_01.append(0)
56         nb_0 += 1
57 print("nb_1 = {}, nb_0 = {}".format(nb_1/(nb_1+nb_0), nb_0/(nb_1+nb_0)))
58
59
60 for i in range(len(liste_01)//(nb_01_interpretation*pas)):
61     liste_nb_aleatoire.append(liste_01_nombre(liste_01, i*nb_01_interpretation*pas, nb_01_interpretation, pas))
62     liste_nb_aleatoire_uniforme.append(liste_nb_aleatoire[i]/(2**nb_01_interpretation))
63
64 print("nb valeur", len(A))
65
66 if sortie_txt:
67     with open(nom_sortie, 'w') as f:
68         for i in range(len(liste_nb_aleatoire_uniforme)):
69             f.write('{}\n'.format(liste_nb_aleatoire_uniforme[i]))
70
71 if sortie_binaire_txt:
72     with open(nom_sortie_binaire_txt, 'w') as f:
73         for i in range(0, len(liste_01), nb_01_interpretation):
74             for j in range(nb_01_interpretation):
75                 try:
76                     f.write('{}'.format(liste_01[i+j]))
77                 except:
78                     pass
79             f.write('\n')
80
81 if sortie_binaire_bin:
82     with open(nom_sortie_binaire_bin, "wb") as f:
83         for i in range(len(liste_nb_aleatoire)):
84             f.write(int(liste_nb_aleatoire[i]).to_bytes(4, "little"))

```